CS/ECE/ISyE 524 — Introduction to Optimization — Spring 2026

March Madness Bracket Optimization

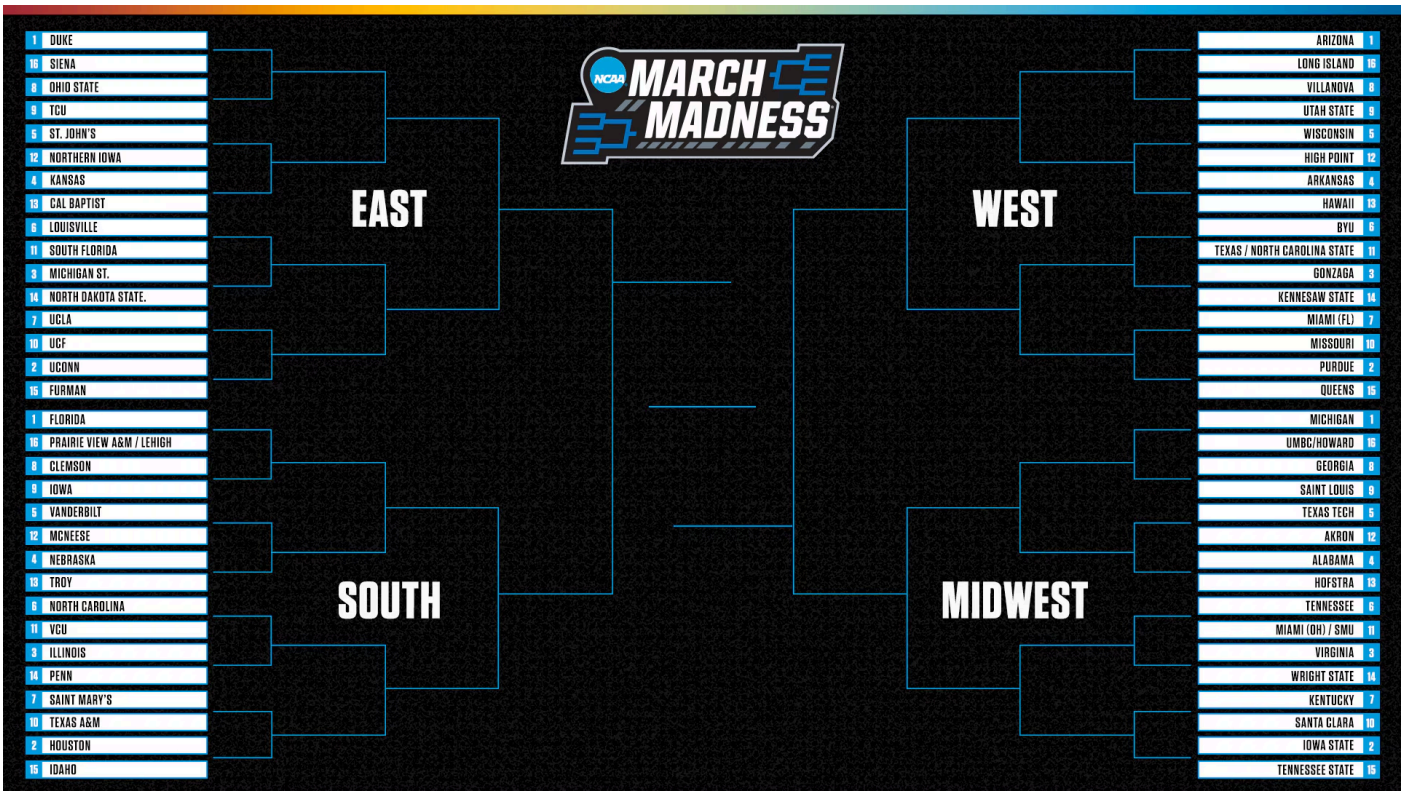
Ryan Logan (rrlogan@wisc.edu) & Tyler White (tkwhite3@wisc.edu)

Table of Contents

1. Introduction
2. Mathematical Model
3. Implementation and Solution
4. Discussion of Results
5. Optional Subsection
6. Conclusion

1. Introduction

March Madness is the annual tournament to determine the best team in college basketball. The tournament field consists of 37 at-large teams and 31 automatic qualifiers who earned their spot by winning their conference tournament, with seeds being determined by a selection committee. After the bracket for the tournament is created, millions of people compete to guess the most correct winners and earn cash prizes or bragging rights. Despite the popularity of the bracket challenge, no perfect bracket has ever been created in the tournaments, with the odds of guessing every matchup correctly being 1 in 9.2 quintillion. In order to aid in the hunt for the perfect bracket, our model will use metrics such as seed, quad 1 wins, and offensive/defensive efficiency to predict the outcome of every matchup. This means that each year the predicted outcomes will not be only tied to choosing the higher seed, yielding a potentially higher success rate in choosing matchups. We will compare this model to the results of previous tournaments to determine whether the model is better than humans at predicting the flow of the tournament. Data can be pulled from NCAA websites and KenPom (for team statistics). We will create a model predicting the outcomes for this years bracket and compare it to the actual results from the tournament. The goal is to create a model that can correctly guess every matchup in the tournament!

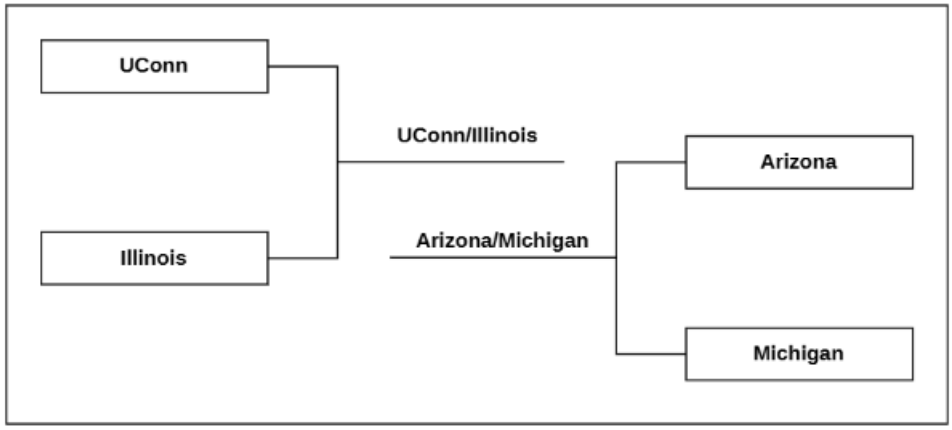


2. Mathematical model

Creating a model predicting outcomes for each game of a 68-team tournament can be a daunting task. To help simplify the problem we will start by creating a model with only 4 teams. We will take the 4 teams that made it to the final four and predict a winner from those teams.

Once we get that model working we will expand our model to predict outcomes for every round of the tournament.

4-team math model



This problem is represented as a network flow problem where the game nodes act as "sinks" with a demand of 1. We will try to maximize the flow. Initially, teams will be assigned a value metric. When two teams meet head-to-head, the team with the greater value metric will win and move on in the tournament (Down the road, the value metric may be recalculated every round to more accurately reflect historical win-loss data).

- $I = \setUconn, Illinois, Arizona, Michigan$
- $G = \setUI, AM, CHAMP =$ the set of the games to be played
- $N =$ the set of all nodes including the teams and games played
- $A =$ the set of arcs representing the bracket path
- $m_i =$ a metric showing the strength of team i
- $d_k =$ the demand at each node where $d_k = 1$ for all $k \in G$
- $x_{ij} =$ the flow on arc (i, j), in this case represents the number of wins a team carries through
- $p_{ij} =$ the points earned in game (i, j)

$$\begin{aligned} \max \quad & \sum_{i \in I} \sum_{(i,j) \in A} x_{ij} * m_i * p_{ij} \\ \text{s.t.} \quad & \sum_{(i,k) \in A} x_{i,k} - \sum_{(k,j) \in A} x_{k,j} = 1 \\ & x_{ir} \geq x_{i(r+1)} \end{aligned}$$

3. Implementation & Solutions

To begin, we will make a simplified model of the 64-team bracket. Let's first consider the case of the Final Four, where there are only four teams remaining and hence three games. For the 2026 tournament, the Final Four was comprised of UConn, Michigan, Arizona, and Illinois.

The criteria for who wins in a head to head will, for this example, be a teams free-throw percentage. This is not a good predictor of success in an actual bracket, but is a metric that is simple enough to use for this example. Of these teams, Illinois shot 78.04%, Michigan shot 74.75%, UConn shot 72.40%, and Arizona shot 73.42%. This data is pulled from the NCAA website.

In [ ]:

In [5]:
```
using JuMP, HiGHS

teams = ["Illinois", "Michigan", "UConn", "Arizona"] # team names
percentages = [78.04, 74.75, 72.40, 73.42]# associated ft percentage
points = [160, 160, 320]


ff = Model(HiGHS.Optimizer) # final four bracket model

# let the demand of each game decision node be 1. Supply of the teams is infinite, for this case only need at most 2.
# the amount of flow out of each team's node will determine how far they go in the bracket
team_nodes = [:UConn, :Illinois, :Arizona, :Michigan]
interior_nodes = [:UI, :AM, :CHAMP]
nodes = vcat(team_nodes, interior_nodes)
interior_demands = Dict(t => 1 for t in interior_nodes)
```

```julia
# make the arcs between nodes
arcs = [(:UConn, :UI), (:Illinois, :UI), (:UI, :CHAMP), (:Arizona, :AM), (:Michigan, :AM), (:AM, :CHAMP)]

# map team names to ft percentage
ftp = Dict(
    :UConn => 72.40,
    :Illinois => 78.04,
    :Arizona => 73.42,
    :Michigan => 74.75
)


# decide the flow of each arc
@variable(ff, flow[arcs] >= 0)

# must meet the demand exactly for every node, aka inflow - outflow = demand
@constraint(ff, demand_met[t in interior_nodes], sum(flow[(i,j)] for (i,j) in arcs if j == t) -
        sum(flow[(i,j)] for (i,j) in arcs if i == t) == interior_demands[t])

# for this case, finding the victors is the same as maximizing the flow out of each team node * associated ft percentage
@objective(ff, Max, sum(sum(flow[(i,j)] for (i,j) in arcs if i == t) * ftp[t] for t in team_nodes))

# solve the model
optimize!(ff)
println()

# print the wins per team
println("Wins per team:")
for t in team_nodes
    println(t, ": ", sum(value(flow[(i,j)]) for (i,j) in arcs if i == t))
end
```

```
Running HiGHS 1.12.0 (git hash: 755a8e027): Copyright (c) 2025 HiGHS under MIT licence terms
LP has 3 rows; 6 cols; 8 nonzeros
Coefficient ranges:
  Matrix  [1e+00, 1e+00]
  Cost    [7e+01, 8e+01]
  Bound   [0e+00, 0e+00]
  RHS     [1e+00, 1e+00]
Presolving model
2 rows, 5 cols, 6 nonzeros  0s
0 rows, 0 cols, 0 nonzeros  0s
Presolve reductions: rows 0(-3); columns 0(-6); nonzeros 0(-8) - Reduced to empty
Performed postsolve
Solving the original LP from the solution after postsolve

Model status        : Optimal
Objective value     :  2.3083000000e+02
P-D objective error :  6.1431092877e-17
HiGHS run time      :          0.00

Wins per team:
UConn: 0.0
Illinois: 2.0
Arizona: 0.0
Michigan: 1.0
```

4. Discussion of Results

4.A. The Final Four Model

Due to the simplicity of the Final Four model, where winners are decided purely based on free-throw percentage, it is trivial to see that Illinois should win everything, and Michigan will get the best of Arizona in a head-to-head. The corresponding optimizer results match this conclusion, with Michigan winning one game and Illinois winning two.

The part of this model that is not as intuitive upon first inspection is how the number of wins is calculated. Because there are no edge capacities, each team (source) node can supply any amount of wins, and each game (interior) node has a demand of exactly 1, meaning that to reach further and further into the interior forces teams to supply more wins. Because the objective value is essentially the number of wins per team times the value of that win in terms of probability, the flow of the edge connecting a team node to their first found game node is equivalent to the number of times they win overall. Based on the number of wins, we can extrapolate how far each team goes in the tournament.

4.A.a. or subsubsections

5. Conclusion

Summarize your findings and your results, and talk about at least one possible future direction; something that might be interesting to pursue as a follow-up to your project.

Progress Report Discussion###

- We currently have the introduction and mathematical model complete. We also have working code for the simplified 4-team model. To complete the project, we need to expand to include all the teams and rounds. This should not take too much time, as we already have the foundation laid. We also need to incorporate the points for the different rounds into our model. This will be a little more challenging because of the way we laid out our original code. Finally, we will come up with some more complex metrics for creating the model to get different results. We will include different models using different metrics so that we can compare.
- One concern we have is that models like this can get very complicated. However, since we started with a very basic model with only 4 teams and a simple metric, it will be doable to slowly add more complicated layers to the model.
- Another potential concern is the difference in the formula for winning between rounds. For example, if proximity to home is used as a factor in the metric, the value will change depending on how far a team goes in the tournament. It is easy enough to make one method of determining winners in a bracket, but harder to re-evaluate between rounds. Due to this, the objective function may become very complicated. This mahy be solvable by using the value of previous edges as a factor in determining inner-layer edge flows.

In [ ]: